Estudante: Haniel Botelho Ribeiro (2023028552)
Curso: Ciências da Computação
Professora: Olga Nikolaevna Goussevskaia
Disciplina: Algoritmos I
Turma: TN
Trabalho Prático 1: Império de Archadia

1. Introdução

Este trabalho aborda a organização militar no contexto do Império de Archadia, governado pelo imperador Gramis Gana Solidor. O imperador solicitou um mapeamento detalhado dos centros urbanos e das estradas que os conectam, visando o planejamento estratégico de segurança das cidades. Dentre os objetivos deste planejamento temos (i) a definição da capital do império como a base militar central, de onde seja possível alcançar todos os centros urbanos na menor distância possível. (ii) identificação de centros urbanos que possam funcionar como batalhões secundários, para garantir que as tropas consigam retornar à capital ou a um ponto de apoio em cenários onde não existam rotas de retorno direto. (iii) mapeamento de rotas de patrulha para cada batalhão, otimizando a cobertura de segurança de Archadia.

2. Modelagem

Este problema pode ser modelado de maneira eficiente utilizando grafos direcionados, onde as **cidades** representam os **vértices** e as **estradas** entre elas são as **arestas**. Para lidar com as diversas operações e algoritmos necessários, a modelagem foi dividida em duas classes principais: **Grafo** e **Estado**.

Classe Grafo

A classe **Grafo** encapsula a estrutura de dados que armazena as cidades e as conexões entre elas. Ela utiliza um vetor de strings para armazenar as cidades e um unordered_map (string, vetor de string) para armazenar as arestas direcionadas saindo de cada cidade, garantindo um armazenamento de complexidade linear.

Os métodos da classe **Grafo** incluem: **add**: Adiciona uma cidade ou uma estrada entre cidades; **get**: Retorna o vetor de cidades ou o vetor de adjacência de uma cidade específica; **has**: Verifica se uma aresta ou cidade existe dentro da estrutura.

Além disso, a classe **Grafo** conta com algoritmos essenciais para a resolução do problema: **BFS**: encontra o caminho entre duas cidades, usado no balanceamento. **BFSDistanciaMaxima**: Retorna a maior estrada a partir de um vértice arbitrário, utilizado na determinação da capital ideal. **DFS**: A busca em profundidade, com uma versão especializada para o grafo transposto, sendo utilizada no algoritmo de Kosaraju para identificar componentes fortemente conexas (CFCs). **identificarExcessos**: Identifica vértices com excesso de entrada ou saída, necessário para balancear o grafo. **balancearGrafo**: Ajusta o grafo para garantir que ele possa ser convertido em um circuito euleriano. **verificarEuleriano**: Verifica se o grafo possui um circuito euleriano, ou seja, se é possível percorrer todas as arestas sem repetições e retornar ao ponto inicial. **hierholzer**: Implementa o algoritmo de Hierholzer, utilizado para encontrar o circuito euleriano em um grafo.

Classe Estado

A classe **Estado** representa a configuração de um império composto por cidades conectadas por estradas, gerenciadas por um grafo direcionado. Essa classe tem como objetivo principal a administração da estrutura do império, utilizando a classe **Grafo** para armazenar as cidades e suas conexões e fornecendo operações como o cálculo da capital do império, a execução de algoritmos de busca, a identificação de componentes fortemente conexas (CFCs) e a definição de rotas de patrulhamento.

Os principais atributos da classe **Estado** são: **Grafo Imperio**: Representa o grafo do império, contendo as cidades e suas conexões; inteiros **centros** e **estradas**; **capital**:

Contém o nome da cidade que será escolhida como capital; **cfcs**: Vetor que armazena as componentes fortemente conexas (CFCs) identificadas pelo algoritmo de Kosaraju; **patrulhamentos**: Lista de patrulhamentos, representados por pares de cidades e as respectivas rotas; **batalhoes**: Contém o número de batalhões necessários fora da capital, bem como os locais onde devem ser posicionados.

Os principais métodos da classe **Estado** são: **encontrarCapital()**: Determina a cidade que será a capital, baseando-se na maior distância máxima de uma cidade arbitrária; **kosaraju()**: Executa o algoritmo de Kosaraju para identificar as componentes fortemente conexas do grafo; **realizarPatrulhamento()**: Organiza rotas de patrulhamento, balanceando o grafo quando necessário para garantir a viabilidade das rotas eulerianas; **printCapital()**: Imprime o nome da cidade capital; **printCFCS()**: Exibe as componentes fortemente conexas identificadas; **printBatalhoes()**: Imprime a quantidade de batalhões e seus respectivos locais; **printPatrulhamento()**: Exibe as rotas de patrulhamento organizadas em circuito euleriano.

Resumo

A classe **Estado** é responsável por gerenciar a rede de cidades e suas estradas, realizar a determinação da capital e organizar patrulhamentos de maneira eficiente. Para isso, ela se apoia na classe **Grafo**, que implementa a estrutura de dados e os algoritmos necessários para manipular e analisar o grafo, como a busca em profundidade e largura, a identificação de componentes fortemente conexas e a construção de circuitos eulerianos. Juntas, as classes **Estado** e **Grafo** oferecem uma modelagem robusta para resolver o problema de gerenciamento de um império, com foco na determinação da capital, na organização dos batalhões e no planejamento das rotas de patrulhamento.

3. **Solução**

Por questão de limite de páginas explicarei os algoritmos mais importantes:
O algoritmo de kosaraju serve para encontrar componentes fortemente conexas, ele utiliza de dfs e dfs no grafo transposto:

Inicialize um mapa visitado para marcar os nós visitados
Inicialize uma pilha ordem para armazenar a ordem de finalização
Inicialize uma lista componentes para armazenar as CFCs encontradas

Primeira fase: Construir a ordem de finalização
Para cada cidade em cidades:
Se cidade não está marcada como visitada:
Chamar dfs(cidade, visitado, ordem)

Criar o grafo transposto

Resetar o estado de visitado
Para cada entrada em visitado:
Marcar todas as cidades como não visitadas

Segunda fase: Encontrar as CFCs no grafo transposto
Enquanto ordem não está vazia:
cidade ← topo da pilha ordem
Remover cidade do topo da pilha

Se cidade não está marcada como visitada:
Inicializar uma lista vazia cfc
Chamar transposto.dfs(cidade, visitado, cfc)
Adicionar cfc à lista componentes

Retornar componentes

VerificarEuleriano: itera sobre as cidades de uma componente fortemente conexa e verifica o grau de saída e entrada, caso não seja igual, configura-se em não euleriano. **identificarExcessos:** retorna dois vetores, um onde tem cidades com excessos de saída outro com excesso de entrada, semelhante ao verificaEuleriano:

```
Crie dois mapas vazios: outDegree e inDegree (contadores de graus)

Para cada cidade em cfc:
Inicialize outDegree[cidade] como 0
Inicialize inDegree[cidade] como 0

Para cada cidade em cfc:
Para cada vizinho na lista de adjacência da cidade:
Se o vizinho pertence a cfc:
        Incrementar outDegree[cidade] em 1
        Incrementar inDegree[vizinho] em 1

Para cada cidade em cfc:

Se outDegree[cidade]> inDegree[cidade]: Adicione cidade ao vetor excessosSaida
Se inDegree[cidade]> outDegree[cidade]: Adicione cidade ao vetor excessosEntrada
//modificação em verifica euleriano
Se inDegree[cidade] != outDegree[cidade] return False
```

O propósito de balancear o grafo é tornar a componente fortemente conexa em um circuito euleriano, ele pega as arestas com excessos de entrada e de saída e acha um caminho entre eles, depois duplica esse caminho. Segue-se o pseudocódigo de **balancearGrafo:**

```
Função balancearGrafo(excessosEntrada, excessosSaida):
Para cada índice i de 0 até tamanho de excessosEntrada - 1:
cidadeEntrada = excessosEntrada[i]
cidadeSaida = excessosSaida[i]

caminho = BFS(cidadeEntrada, cidadeSaida)

Se caminho não for vazio:
Para cada índice j de 0 até tamanho do caminho - 2:
        addEstrada(caminho[j], caminho[j + 1])
```

O algoritmo hierholzer encontra um caminho em um circuito euleriano, e funciona da seguinte forma:

```
Crie uma cópia da lista de adjacência chamada adjCopy
Crie uma pilha vazia chamada pilha
Crie um vetor vazio chamado circuito
Defina a start como o primeiro elemento de cfc e candidato como uma string vazia
Empilhe start na pilha
Enquanto a pilha não estiver vazia:
cidade ← topo da pilha
Se adjCopy[cidade] não estiver vazia:
        candidato ← último elemento de adjCopy[cidade]
        Remova o último elemento de adjCopy[cidade]
        Se candidato pertence a cfc:
                Empilhe candidato na pilha
Se adjCopy[cidade] está vazia:
        Adicione cidade ao vetor circuito
        Desempilhe o topo da pilha
Retorne circuito
```

A função encontrar capital de estado, procura a bfs máxima de todas as cidades do grafo, ou seja, complexidade temporal de O(V*O(V+E)). A função kosaraju chama kosaraju do grafo e armazena as CFCs, para identificar os batalhões. A função realizaPatrulhamento faz uma chamada de Hierholzer para cada CFCs.

4. Análise de Complexidade

BFS busca o caminho mais curto para todos os vértices, a cada iteração adiciona todas as arestas de um vértice, implementando lista de adjacência temos complexidade de tempo O(V+E) e espacial O(V+E). A DFS tem complexidade igual, de forma análoga ela faz uma busca em profundidade indo o mais longe possível a partir de cada novo vértice. A complexidade de transpor um grafo também é a mesma, pois percorre a lista de adjacência de todos os vértices e inverte-os.

Kosaraju utiliza dfs duas vezes e transpor o grafo uma vez, ou seja, possui complexidade temporal e espacial O(V+E). A complexidade de tempo ao identificar excessos, primeiro itera sobre todos os vértices da CFC atribuindo grau zero: O(V); depois itera sobre cada lista de adjacência de cada cidade do CFC verificando se a estrada leva para dentro do CFC, O(E* O(V)) ≃ O(V*E); depois itera para cada Cidade verificando se tem excesso de entrada ou saída, O(V): resultando em O(V) + O(V*E) + O(V) ≃ O(V*E), o retorno são vetores de excesso com complexidade espacial adicional de O(V) + custo de armazenar o grau de cada vértice O(V). Verificar Euleriano segue a mesma lógica, mas retorna booleano.

A complexidade temporal de balancear um grafo se dá: pelo loop externo (n = quantidade de excessos) caminho recebe BFS(InExcesso,OutExcesso): O(n*O(V+E)), depois para cada aresta do caminho duplicar elas: O(n*O(E)) ≃ O(n*(V+E)), complexidade espacial é O(E) pois no pior caso duplica todas as arestas. O custo espacial na iniciação de hierholzer é O(E) pois copia as arestas + uma pilha, um vetor e uma string. Dentro do loop adiciona na pilha o último vizinho de uma cidade O(1) + remover ele da lista O(1) + verificar se está na CFC O(V). O loop itera sobre todas as arestas da CFC, logo temos O(E*O(1+V)), custo adicional para garantir consistência.

5. Considerações Finais

Este trabalho envolveu uma análise crítica para entender o problema e adaptar algoritmos de grafos já conhecidos às necessidades específicas do contexto. Um dos maiores desafios foi o balanceamento do grafo, pois os algoritmos existentes tradicionalmente funcionam em grafos eulerianos. Além disso, a criação de casos de teste que pudessem contornar o algoritmo implementado foi um exercício criativo que impulsionou melhorias no programa, tornando-o mais robusto e capaz de lidar com essas situações excepcionais. Em conclusão, foi possível compreender de forma prática como os algoritmos aprendidos se aplicam a problemas do mundo real, ampliando a visão além de sua aplicação teórica.

6. Referências

Aulas da professora Olga.
Colegas de classe.